ĐẠI HỌC QUỐC GIA, THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



KIẾN TRÚC MÁY TÍNH (CO2008)

BÁO CÁO LAB 2
Branches and procedures

Sinh viên: Hoàng Đức Tài - 2510219

TP. HỒ CHÍ MINH, THÁNG 11 NĂM 2026

Mục lục

1 Exercise 1 **1**

2 Exercise 2 **2**

3 Exercise 3 **4**

4 Exercise 4 **6**

1 Exercise 1

Mảng 5 phần tử khai báo bằng `.word 0:5`. `$s0` giữ địa chỉ đầu mảng, `$t0` là chỉ số phần tử đang xử lý:

```
  array: .word 0:5

  la $s0, array     # địa chỉ đầu mảng
  li $t0, 0         # chỉ số index
```

Mỗi vòng lặp đọc một số nguyên, chuyển số đó về bội số của 4 gần nhất rồi lưu vào mảng.

Phần dư lấy bằng `andi $t1, $v0, 3`. Chia cho $4 = 2^2$ là bỏ đi hai bit thấp, nên phần dư nằm gọn trong hai bit đó; AND với $3 = 11_2$ giữ lại đúng hai bit ấy và xoá phần còn lại. Cách này chỉ cần một lệnh, không dùng đến `HI/LO` như `div`, và vì là bitwise nên số âm cũng ra phần dư 0–3 như phép chia dư không âm.

Phần dư 0 thì giữ nguyên; dư 1 hoặc 2 thì trừ đi phần dư (làm tròn xuống); dư 3 thì cộng thêm 1 (làm tròn lên). Vì thế 35 thành 36, còn 34 và 33 thành 32.

```
process_loop:
  li $v0, 5                 # syscall 5: đọc số nguyên
  syscall

  andi $t1, $v0, 3          # $t1 = n % 4
  sub $v0, $v0, $t1         # n -= n % 4: làm tròn xuống
  beq $t1, 3, round_up      # dư 3 thì bội số gần nhất nằm phía trên
  j next

round_up:
  addi $v0, $v0, 4          # n += 4: nhảy lên bội số kế tiếp

next:
  sll $t1, $t0, 2           # $t1 = 4i
  add $t1, $t1, $s0         # $t1 = base + 4i
  sw $v0, 0($t1)            # lưu vào phần tử thứ i
  addi $t0, $t0, 1          # i++
  bne $t0, 5, process_loop # đủ 5 phần tử thì dừng
```

Vòng lặp thứ hai đọc lại mảng và in từng phần tử, mỗi số cách nhau một dấu cách:

```
  li $t0, 0          # i = 0: duyệt lại từ đầu mảng
print:
  li $v0, 1          # syscall 1: in số nguyên
  sll $a0, $t0, 2    # $a0 = 4i
  add $a0, $s0, $a0 # $a0 = base + 4i
```

```
  lw $a0, 0($a0)     # $a0 = phần tử thứ i
  syscall

  li $v0, 11         # syscall 11: in ký tự
  li $a0, ' '        # in dấu cách
  syscall

  addi $t0, $t0, 1   # i++
  bne $t0, 5, print
```

Chương trình không in lời nhắc khi đọc, nên nhập lần lượt 35, 34, 33, 12, 7 thì in ra:

```
36 32 32 12 8
```

2 Exercise 2

Các chuỗi trong `.data` đảm nhiệm lời nhắc, thông báo lỗi và hai chuỗi kết quả:

```
prompt_a: .asciiz "a = "
prompt_b: .asciiz "b = "
err_msg: .asciiz "Input must be a positive integer. Please try again.\n"
gcd_output: .asciiz "GCD = "
lcm_output: .asciiz "\nLCM = "
```

Mỗi số đọc vào được kiểm tra bằng `bgtz`: lớn hơn 0 thì sang nhãn `valid_a` lưu vào thanh ghi, ngược lại in `err_msg` rồi quay lại chính nhãn đọc để hỏi lại. Hai khối đọc giống nhau hoàn toàn, chỉ khác chuỗi lời nhắc và thanh ghi đích:

```
read_a:
  li $v0, 4          # syscall 4: in chuỗi
  la $a0, prompt_a   # in "a = "
  syscall

  li $v0, 5          # syscall 5: đọc số nguyên
  syscall
  bgtz $v0, valid_a # a > 0 thì hợp lệ

  li $v0, 4          # syscall 4: in chuỗi
  la $a0, err_msg    # thông báo nhập sai
  syscall

  j read_a           # hỏi lại từ đầu

valid_a:
```

```
  move $s0, $v0      # $s0 = a
```

GCD tính bằng vòng lặp Euclid: mỗi bước thay cặp (a, b) bằng $(b, a \bmod b)$, dừng khi $b = 0$ thì a là GCD. Hai thanh ghi `$t0` và `$t1` đóng vai trò a và b, phần dư lấy qua `div` + `mfhi`:

```
gcd:
  move $t0, $s0           # $t0 = a
  move $t1, $s1           # $t1 = b

gcd_loop:
  beqz $t1, print_output # b == 0: $t0 là GCD

  div $t0, $t1            # chia a cho b
  mfhi $t2                # $t2 = a % b

  move $t0, $t1           # a = b
  move $t1, $t2           # b = a % b

  j gcd_loop              # lặp lại với cặp mới
```

LCM tính từ GCD theo $\text{lcm}(a, b) = a \times b / \gcd(a, b)$, ngay tại nhãn `print_output` với `mul` rồi `div` + `mflo`:

```
print_output:
  mul $t1, $s0, $s1 # $t1 = a * b
  div $t1, $t0      # chia cho GCD
  mflo $t1          # $t1 = LCM
```

Kết quả được in ra:

```
  li $v0, 4           # syscall 4: in chuỗi
  la $a0, gcd_output # in "GCD = "
  syscall

  li $v0, 1           # syscall 1: in số nguyên
  move $a0, $t0       # in GCD
  syscall

  li $v0, 4           # syscall 4: in chuỗi
  la $a0, lcm_output # in "\nLCM = "
  syscall

  li $v0, 1           # syscall 1: in số nguyên
  move $a0, $t1       # in LCM
  syscall
```

Nhập $a = 6$, $b = 45$:

```
a = 6
b = 45
GCD = 3
LCM = 90
```

Nhập $a = -5$ (bị từ chối), sau đó $a = 6$, $b = 45$:

```
a = -5
Input must be a positive integer. Please try again.
a = 6
b = 45
GCD = 3
LCM = 90
```

3 Exercise 3

Mảng 10 phần tử và ba thanh ghi lưu trạng thái: `$s1` là giá trị lớn nhất, `$s2` là giá trị lớn thứ hai, `$s3` là cờ báo đã có giá trị lớn thứ hai hay chưa. Phần tử đầu tiên được lưu vào mảng và cũng làm mốc khởi tạo cho `$s1`:

```
  la $s0, arr                # $s0 = địa chỉ mảng
  li $s1, 0                  # $s1 = giá trị lớn nhất
  li $s2, 0                  # $s2 = giá trị lớn thứ hai
  li $s3, 0                  # $s3 = 1 khi đã có giá trị lớn thứ hai
  li $t0, 0                  # $t0 = chỉ số i

input_loop:
  beq $t0, 10, print_second # đủ 10 phần tử thì in kết quả

  li $v0, 4                  # syscall 4: in chuỗi
  la $a0, prompt1            # in "Please insert a element: "
  syscall

  li $v0, 5                  # syscall 5: đọc số nguyên
  syscall

  sll $t1, $t0, 2            # $t1 = 4i
  add $t1, $t1, $s0          # $t1 = base + 4i
  sw $v0, 0($t1)             # lưu vào phần tử thứ i

  beqz $t0, first_input      # phần tử đầu chưa có mốc so sánh
```

Sau khi đã có phần tử đầu, mỗi số đọc vào rơi vào bốn trường hợp dưới đây. Số bằng `$s1`

bị bỏ qua vì nó cũng là giá trị lớn nhất, không phải giá trị lớn thứ hai:

```
  bgt $v0, $s1, new_max     # lớn hơn max: max cũ thành second
  beq $v0, $s1, next        # bằng max: bỏ qua
  beqz $s3, new_second      # chưa có second: nhận luôn
  bgt $v0, $s2, new_second # lớn hơn second: thay second
  j next

first_input:
  move $s1, $v0             # phần tử đầu làm max
  j next

new_max:
  move $s2, $s1             # second = max cũ
  move $s1, $v0             # max = phần tử mới
  li $s3, 1                 # đã có second
  j next

new_second:
  move $s2, $v0             # second = phần tử mới
  li $s3, 1                 # đã có second
  j next
```

Chương trình in giá trị lớn thứ hai trước, sau đó là một vòng quét thứ hai tìm mọi chỉ số có phần tử bằng `$s2`. Trong vòng quét đó, `$t1` là cờ cho biết đã in chỉ số nào chưa: nó khởi tạo bằng 0, chỉ số khớp đầu tiên được in một mình rồi cờ bật lên 1; từ chỉ số thứ hai trở đi mới in thêm `", "` phía trước. Nhờ vậy kết quả không có dấu phẩy thừa ở đầu:

```
print_second:
  beqz $s3, no_second      # chưa có second: báo không tồn tại

  li $v0, 4                # syscall 4: in chuỗi
  la $a0, msg1             # chuỗi "Second largest value is "
  syscall

  li $v0, 1                # syscall 1: in số nguyên
  move $a0, $s2            # in giá trị lớn thứ hai
  syscall

  li $v0, 4                # syscall 4: in chuỗi
  la $a0, msg2             # chuỗi ", found in index "
  syscall

  li $t0, 0                # $t0 = chỉ số i
  li $t1, 0                # $t1 = 1 khi đã in ít nhất một chỉ số
```

```
find_second_indexes:
  beq $t0, 10, exit        # quét hết mảng thì kết thúc

  sll $t2, $t0, 2          # $t2 = 4i
  add $t2, $t2, $s0        # $t2 = base + 4i
  lw $t3, 0($t2)           # $t3 = phần tử thứ i

  bne $t3, $s2, find_next # không bằng second: bỏ qua
  beqz $t1, print_idx      # chỉ số đầu tiên: in không dấu phẩy

  li $v0, 4                # syscall 4: in chuỗi
  la $a0, msg3             # in ", " trước các chỉ số sau
  syscall

print_idx:
  li $v0, 1                # syscall 1: in số nguyên
  move $a0, $t0            # in chỉ số
  li $t1, 1                # đánh dấu đã in ít nhất một chỉ số
  syscall

find_next:
  addi $t0, $t0, 1         # i++
  j find_second_indexes    # tiếp tục quét
```

Khi cờ `$s3` vẫn là 0, nhãn `no_second` in thông báo rồi thoát:

```
no_second:
  li $v0, 4     # syscall 4: in chuỗi
  la $a0, msg5 # "No second largest value exists."
  syscall
  j exit
```

Nhập $1, 2, 7, -7, 3, 7, 4, 6, 8, 7$:

```
Second largest value is 7, found in index 2, 5, 9.
```

Với mảng toàn số bằng nhau, cờ `$s3` không bao giờ được bật. Nhập $5, 5, 5, 5, 5, 5, 5, 5, 5, 5$:

```
No second largest value exists.
```

4 Exercise 4

Chương trình đọc một số nguyên, đưa giá trị sang `$a0` làm tham số cho `factorial` và kiểm tra hai trường hợp trước khi gọi. Ngưỡng 12 là giới hạn của số nguyên 32 bit có dấu: $12! = 479\,001\,600$ vẫn nằm dưới $2^{31} - 1 = 2\,147\,483\,647$, còn $13! = 6\,227\,020\,800$ đã

vượt:

```
input:
  li $v0, 5              # syscall 5: đọc số nguyên
  syscall

  move $a0, $v0          # $a0 = n: tham số cho factorial
  bltz $a0, error        # n < 0: không có giai thừa
  bgt $a0, 12, overflow # n > 12: kết quả vượt 32 bit
```

Kết quả trả về trong `$v0` được lưu sang `$t0` rồi in ra:

```
  jal factorial
  move $t0, $v0 # $t0 = n!

  li $v0, 1      # syscall 1: in số nguyên
  move $a0, $t0 # in n!
  syscall

  j exit
```

Giai thừa viết đệ quy theo $n! = n \times (n-1)!$, dừng ở $0! = 1$. Sau lời gọi `jal factorial`, `$a0` đã bị tầng dưới sửa, nên giá trị n của tầng hiện tại phải lưu vào stack cùng với `$ra`. Khi tầng dưới trả về, `$v0` đang giữ $(n-1)!$, nhân với n vừa khôi phục để được kết quả:

```
factorial:
  beqz $a0, base_case # 0! = 1

  addi $sp, $sp, -4    # mở slot cho $ra
  sw $ra, 0($sp)       # lưu địa chỉ trả về
  addi $sp, $sp, -4    # mở slot cho n
  sw $a0, 0($sp)       # lưu n

  addi $a0, $a0, -1    # tham số mới: n - 1
  jal factorial        # $v0 = (n-1)!

  lw $t0, 0($sp)       # lấy lại n
  addi $sp, $sp, 4     # đóng slot của n
  lw $ra, 0($sp)       # khôi phục địa chỉ trả về
  addi $sp, $sp, 4     # đóng slot của $ra

  mul $v0, $v0, $t0    # n! = n * (n-1)!
  jr $ra

base_case:
  li $v0, 1            # 0! = 1
```

```
  jr $ra
```

Hai nhãn lỗi chỉ in thông báo tương ứng rồi nhảy tới `exit`:

```
error:
  li $v0, 4     # syscall 4: in chuỗi
  la $a0, msg1 # "A factorial for your number can not be found."
  syscall
  j exit

overflow:
  li $v0, 4     # syscall 4: in chuỗi
  la $a0, msg2 # "Number is too large for 32-bit integer."
  syscall
  j exit
```

Nhập 5, chương trình in:

```
120
```

Nhập -2, giá trị âm rẽ sang nhãn `error`:

```
A factorial for your number can not be found.
```

Nhập 13, giá trị vượt ngưỡng nên rẽ sang nhãn `overflow`:

```
Number is too large for 32-bit integer.
```